



Practical Report - <07>
SE3082 - Parallel Computing
3rd year 1st semester - 2026

Student IDs: IT24104152

Exercise 01

```
ubuntu@ip-172-31-27-189: /shared
ubuntu@ip-172-31-27-189:/shared$ mpirun -np 4 --hostfile /shared/machinefile /shared/sum_bcast
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 250000) => local_sum = 31250125000
Rank 1: summed indices [250000, 500000) => local_sum = 93750125000
Rank 2: summed indices [500000, 750000) => local_sum = 156250125000
Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000

[Bcast] Total sum    = 500000500000
[Bcast] Expected    = 500000500000
[Bcast] Correct?    = YES
[Bcast] Time        = 0.0317 sec
ubuntu@ip-172-31-27-189:/shared$
```

The root process initializes the complete array and broadcasts it to all processes using `MPI_Bcast`. Each process calculates the sum of its assigned portion, and the partial sums are sent back to the root using `MPI_Send` and `MPI_Recv`. The final result matches the expected total of 500000500000.

Exercise 02

```
ubuntu@ip-172-31-27-189:/shared$ mpirun -np 4 --hostfile /shared/machinefile /shared/sum_scatter
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000
Rank 0: local_sum = 31250125000
Rank 3: local_sum = 218750125000

[Scatter] Total sum = 500000500000
[Scatter] Expected  = 500000500000
[Scatter] Correct?  = YES
[Scatter] Time      = 0.005216 sec
Rank 2: local_sum = 156250125000
ubuntu@ip-172-31-27-189:/shared$
```

In this version, `MPI_Scatter` is used to distribute only the required section of the array to each process instead of sending the full array to every process. Each process calculates its local sum, and the partial results are returned to the root using `MPI_Send` and `MPI_Recv`.

Exercise 03

```
ubuntu@ip-172-31-27-189:/shared$ mpirun -np 4 --hostfile /shared/machinefile /shared/sum_gather
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000
Rank 0: local_sum = 31250125000
Rank 2: local_sum = 156250125000
Rank 3: local_sum = 218750125000

[Gather] Total sum = 500000500000
[Gather] Expected = 500000500000
[Gather] Correct? = YES
[Gather] Time = 0.004903 sec
```

MPI_Gather replaces the manual MPI_Send and MPI_Recv operations used to collect partial sums. Each process sends its local_sum to the root, where all partial sums are stored in an array and then added together to produce the final result.

Exercise 04

```
ubuntu@ip-172-31-27-189:/shared$ mpirun -np 4 --hostfile /shared/machinefile /shared/sum_reduce
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000
Rank 0: local_sum = 31250125000
Rank 2: local_sum = 156250125000
Rank 3: local_sum = 218750125000

[Reduce] Total sum = 500000500000
[Reduce] Expected = 500000500000
[Reduce] Correct? = YES
[Reduce] Time = 0.006467 sec
ubuntu@ip-172-31-27-189:/shared$ |
```

MPI_Reduce combines all local sums automatically using the MPI_SUM operation. Unlike MPI_Gather, there is no need for the root process to manually add the partial results. The final total is available only on the root process.

Exercise 05

```
ubuntu@ip-172-31-27-189:/shared$ mpirun -np 4 --hostfile /shared/machinefile /shared/sum_allreduce
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, total_sum = 500000500000, contribution = 6.25%

[Allreduce] Total sum = 500000500000
[Allreduce] Expected = 500000500000
[Allreduce] Correct? = YES
[Allreduce] Time = 0.004577 sec
Rank 2: local_sum = 156250125000, total_sum = 500000500000, contribution = 31.25%
Rank 1: local_sum = 93750125000, total_sum = 500000500000, contribution = 18.75%
Rank 3: local_sum = 218750125000, total_sum = 500000500000, contribution = 43.75%
ubuntu@ip-172-31-27-189:/shared$ |
```

MPI_Allreduce performs the same summation operation as MPI_Reduce, but the final total is made available to every process. This allows each process to calculate and display the percentage contribution of its local sum to the overall total.

Exercise 06

```
ubuntu@ip-172-31-27-189:/shared$ mpicc -O2 sum_scan.c -o sum_scan
ubuntu@ip-172-31-27-189:/shared$ mpirun -np 4 --hostfile /shared/machinefile /shared/sum_scan
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, prefix_sum = 31250125000, sum_before_me = 0
Rank 1: local_sum = 93750125000, prefix_sum = 125000250000, sum_before_me = 31250125000
Rank 3: local_sum = 218750125000, prefix_sum = 500000500000, sum_before_me = 281250375000

[Scan] Total sum = 500000500000
Rank 2: local_sum = 156250125000, prefix_sum = 281250375000, sum_before_me = 125000250000
[Scan] Expected = 500000500000
[Scan] Correct? = YES
[Scan] Time = 0.005240 sec
ubuntu@ip-172-31-27-189:/shared$ |
```

MPI_Scan performs a prefix reduction, where each process receives the cumulative sum of all ranks up to its own rank. Therefore, the last process obtains the total sum of the complete array, while earlier processes receive progressively increasing partial totals.

Exercise 07

Program	Distribution	Result collection	Full array on every process?	Manual root sum?	Final result available
sum_bcast	MPI_Bcast	MPI_Send/Recv	Yes	Yes	Root
sum_scatter	MPI_Scatter	MPI_Send/Recv	No	Yes	Root
sum_gather	MPI_Scatter	MPI_Gather	No	Yes	Root
sum_reduce	MPI_Scatter	MPI_Reduce	No	No	Root
sum_allreduce	MPI_Scatter	MPI_Allreduce	No	No	All ranks
sum_scan	MPI_Scatter	MPI_Scan	No	No	Different cumulative result per rank

Exercise 08

```
ubuntu@ip-172-31-27-189:/shared$ make run
mpicc -O2 sum_bcast.c -o sum_bcast
mpirun -np 4 --hostfile /shared/machinefile ./sum_bcast
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 250000) => local_sum = 31250125000
Rank 1: summed indices [250000, 500000) => local_sum = 93750125000
Rank 2: summed indices [500000, 750000) => local_sum = 156250125000
Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000

[Bcast] Total sum = 500000500000
[Bcast] Expected = 500000500000
[Bcast] Correct? = YES
[Bcast] Time = 0.0308 sec
mpirun -np 4 --hostfile /shared/machinefile ./sum_scatter
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000
Rank 0: local_sum = 31250125000

[Scatter] Total sum = 500000500000
[Scatter] Expected = 500000500000
[Scatter] Correct? = YES
[Scatter] Time = 0.005603 sec
Rank 2: local_sum = 156250125000
Rank 3: local_sum = 218750125000
mpirun -np 4 --hostfile /shared/machinefile ./sum_gather
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000
Rank 1: local_sum = 93750125000

[Gather] Total sum = 500000500000
[Gather] Expected = 500000500000
[Gather] Correct? = YES
[Gather] Time = 0.004838 sec
Rank 2: local_sum = 156250125000
Rank 3: local_sum = 218750125000
mpirun -np 4 --hostfile /shared/machinefile ./sum_reduce
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000
Rank 0: local_sum = 31250125000
Rank 3: local_sum = 218750125000
Rank 2: local_sum = 156250125000

[Reduce] Total sum = 500000500000
[Reduce] Expected = 500000500000
[Reduce] Correct? = YES
[Reduce] Time = 0.006217 sec
mpirun -np 4 --hostfile /shared/machinefile ./sum_allreduce
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, total_sum = 500000500000, contribution = 6.25%

[Allreduce] Total sum = 500000500000
[Allreduce] Expected = 500000500000
[Allreduce] Correct? = YES
[Allreduce] Time = 0.005062 sec
Rank 1: local_sum = 93750125000, total_sum = 500000500000, contribution = 18.75%
Rank 2: local_sum = 156250125000, total_sum = 500000500000, contribution = 31.25%
Rank 3: local_sum = 218750125000, total_sum = 500000500000, contribution = 43.75%
mpirun -np 4 --hostfile /shared/machinefile ./sum_scan
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, prefix_sum = 31250125000, sum_before_me = 0
Rank 1: local_sum = 93750125000, prefix_sum = 125000250000, sum_before_me = 31250125000
Rank 2: local_sum = 156250125000, prefix_sum = 281250375000, sum_before_me = 125000250000
Rank 3: local_sum = 218750125000, prefix_sum = 500000500000, sum_before_me = 281250375000

[Scan] Total sum = 500000500000
[Scan] Expected = 500000500000
[Scan] Correct? = YES
[Scan] Time = 0.006944 sec
ubuntu@ip-172-31-27-189:/shared$
```